

## Práctica 4.4 Documentación de Código (Clases) (CE4.g & CE4.i)



Nombre: Mateo Daniel Ortuño Ovando  
Asignatura: Entornos de desarrollo

| fecha de inicialización: 18/02/2026  
| fecha de finalización: 22/02/2026

# ÍNDICE

- 1. INTRODUCCIÓN ..... Pág. 3
  
- 2. DOCUMENTACIÓN DEL CÓDIGO FUENTE ..... Pág. 4
  - 2.1. Interfaz Animal.java
  - 2.2. Clase Gato.java
  - 2.3. Clase Perro.java
  - 2.4. Clase Ejercicio1UD5ED.java (Main)
  
- 3. GENERACIÓN DE JAVADOC (HTML) ..... Pág. 15
  - 3.1. Proceso de generación
  - 3.2. Vista previa del portal de documentación
  
- 4. CONCLUSIÓN ..... Pág. 20

## 1.INTRODUCCION

\_ El presente documento recoge el proceso de documentación y estandarización del proyecto Java "ejercicio1ud5ed".

El objetivo de la práctica consiste en aplicar buenas prácticas de ingeniería de software mediante el uso de Javadoc. Para ello, se ha trabajado sobre una estructura de programación orientada a objetos compuesta por una interfaz (Animal) y sus implementaciones correspondientes (Perro y Gato).

Las tareas principales realizadas han sido:

- ✓ Identificación de elementos: Análisis de clases, métodos y atributos.
- ✓ Etiquetado técnico: Inserción de etiquetas @author, @param, @return y @version para cumplir con el estándar de Java.
- ✓ Gestión de código heredado: Marcado de métodos obsoletos mediante la etiqueta @deprecated.
- ✓ Generación de recursos: Exportación de toda la información técnica a un formato web (HTML) navegable para facilitar el mantenimiento del proyecto.

## 2.DOCUMENTACION DEL CODIGO FUENTE.

\_A continuacion vamos a explicar cada fichero de nuestro proyecto y como hemos realizado la documentacion

### 2.1. Interfaz Animal.java.

\_fichero de Animal.java:

```
/*  
    • Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt  
      to change this license  
    • Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Interface.java to  
      edit this template  
*/  
package ejercicio1ud5ed; /*  
    • Interfaz que define el comportamiento básico de un objeto de tipo Animal.  
    • Proporciona la estructura para obtener la identificación y la comunicación  
      sonora del animal.  
    •  
    • @author Mateo Daniel Ortuño Ovando (mateoortuno11@gmail.com)  
    • @version 1.0  
*/  
public interface Animal { /* * Recupera el nombre asignado al animal. * * @return  
{@code String} que representa el nombre del animal. */  
  
String getNombre();  
  
/**  
 * Genera una representación textual del sonido característico que emite el animal.  
 *  
 * @return {@code String} con la onomatopeya o descripción del sonido del animal.  
 */  
  
String hacerSonido();  
  
}
```

\_En este fichero java hemos aplicado @author / @version: Definición de autoría y revisión del archivo.

\_Método getNombre(): Especifica que retorna la identidad del animal.

\_Método hacerSonido(): Define la acción de emitir sonido.

-Imágenes del código:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Interface.java to edit this template
 */
package ejerciciolud5ed;

/**
 * Interfaz que define el comportamiento básico de un objeto de tipo Animal.
 * Proporciona la estructura para obtener la identificación y la comunicación sonora del animal.
 *
 * @author Mateo Daniel Ortuño Ovando (mateoortuno11@gmail.com)
 * @version 1.0
 */
public interface Animal {

    /**
     * Recupera el nombre asignado al animal.
     *
     * @return {@code String} que representa el nombre del animal.
     */
    String getNombre();

    /**
     * Genera una representación textual del sonido característico que emite el animal.
     *
     * @return {@code String} con la onomatopeya o descripción del sonido del animal.
     */
}
```

## 2.2. Clase Gato.java.

\_ fichero de Gato.java:

```
/**
```

- Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
- Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template

```
/ package ejercicio1ud5ed; /**
```

- Clase que representa a un Gato e implementa la interfaz Animal.
- Define el comportamiento específico y las propiedades de un felino doméstico.
- 
- @author Mateo Daniel Ortuño Ovando ([mateoortuno11@gmail.com](mailto:mateoortuno11@gmail.com))
- @version 1.0

```
*/ public class Gato implements Animal {
```

```
/** El nombre que identifica al gato */
```

```
private String nombre;
```

```
/** Estado que indica si el gato está emitiendo un ronroneo */
```

```
private boolean ronronea;
```

```
/**  
 * Constructor para crear un nuevo Gato.  
 * Por defecto, el gato se crea sin ronronear.  
 *  
 * @param nombre El nombre que se le asignará al gato.  
 */
```

```
public Gato(String nombre) {  
    this.nombre = nombre;  
    this.ronronea = false;  
}
```

```
/**  
 * Recupera el nombre del gato.  
 *  
 * @return {@code String} con el nombre del animal.  
 */
```

```
@Override  
public String getNombre() {  
    return nombre;  
}
```

```
/**  
 * Permite modificar el estado de ronroneo del gato.  
 *  
 * @param ronronea {@code true} para activar el ronroneo, {@code false} para  
desactivarlo.  
 */
```

```
public void setRonronea(boolean ronronea) {  
    this.ronronea = ronronea;  
}
```

```
/**  
 * Emite el sonido característico del gato.  
 *  
 * @return El texto "Miau".  
 */
```

```
@Override  
public String hacerSonido() {  
    return "Miau";  
}
```

```

/**
 * Comprueba si el gato está ronroneando actualmente.
 *
 * @return {@code true} si está ronroneando, {@code false} en caso contrario.
 */
public boolean isRonroneando() {
    return this.ronronea;
}

}

```

-Imágenes del código:

The screenshot shows a code editor with the following Java code:

```

/**
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ejerciciolud5ed;

/**
 * Clase que representa a un Gato e implementa la interfaz Animal.
 * Define el comportamiento específico y las propiedades de un felino doméstico.
 *
 * @author Mateo Daniel Ortuño Ovando (mateoortunoll@gmail.com)
 * @version 1.0
 */
public class Gato implements Animal {

    /** El nombre que identifica al gato */
    private String nombre;

    /** Estado que indica si el gato está emitiendo un ronroneo */
    private boolean ronronea;

    /**
     * Constructor para crear un nuevo Gato.
     * Por defecto, el gato se crea sin ronronear.
     *
     * @param nombre El nombre que se le asignará al gato.
     */
}

```

At the bottom right of the editor, there is a watermark that says "Activar Windows" and "Ve a Configuración para activar Windows."

\_Campos: Documentados brevemente para explicar qué almacenan (nombre y ronronea).

\_Constructor: Explica la inicialización del objeto.

\_Getters/Setters: Especifican claramente qué valor manejan y qué devuelven.

## 2.3. Clase Perro.java

\_fichero de Perro.java:

```
/*  
    • Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt  
      to change this license  
    • Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit  
      this template  
*/  
  
/package ejercicio1ud5ed; /*  
    • Clase que representa a un Perro e implementa la interfaz Animal.  
    • Define el comportamiento de un canino, incluyendo su estado de ánimo y  
      sonidos.  
    •  
    • @author Mateo Daniel Ortuño Ovando (mateoortuno11@gmail.com)  
    • @version 1.0  
*/  
public class Perro implements Animal {  
  
    /** El nombre del perro */  
    private String nombre;  
  
    /** Indica si el perro tiene un comportamiento agresivo o molesto en el momento */  
    private boolean enfadado;  
  
    /** Atributo para almacenar el tamaño del perro. @deprecated Este campo ya no se  
        utiliza. */  
    private String talla;  
  
    /**  
     * Constructor para la clase Perro.  
     * Inicializa al perro con un nombre y en estado no enfadado por defecto.  
     */  
    /**  
     * @param nombre Nombre que recibirá el perro.  
     */  
    public Perro(String nombre) {  
  
        this.nombre = nombre;  
        this.enfadado = false;  
    }  
    /**  
     * Obtiene el nombre del perro.
```



```

*
* @return El nombre almacenado en el atributo {@code nombre}.
*/
@Override
public String getNombre() {
    return nombre;
}

/**
 * Consulta si el perro se encuentra enfadado.
 *
 * @return {@code true} si está enfadado, {@code false} de lo contrario.
 */
public boolean isEnfadado() {
    return enfadado;
}

/**
 * Actualiza el estado de enfado del perro.
 *
 * @param enfadado Nuevo estado de ánimo del perro.
 */
public void setEnfadado(boolean enfadado) {
    this.enfadado = enfadado;
}

/**
 * Emite el sonido característico de un perro.
 *
 * @return La cadena "Guau guau".
 */
@Override
public String hacerSonido() {
    return "Guau guau";
}

/**
 * Determina si el perro está ladrando basándose en su nivel de enfado.
 *
 * @return {@code true} si el perro está enfadado (y por tanto ladra).
 */
public boolean isLadrando() {
    return this.enfadado;
}

```

```
}
```

```
/**
```

```
 * Obtiene la talla del perro.
```

```
 *
```

```
 * @return La talla actual.
```

```
 * @deprecated Este método ya no se usa en la lógica actual del proyecto.
```

```
 */
```

```
public String getTalla() {
```

```
    return this.talla;
```

```
}
```

```
/**
```

```
 * Establece una nueva talla para el perro.
```

```
 *
```

```
 * @param nuevaTalla Texto con la nueva talla.
```

```
 * @deprecated Este método ya no se usa en la lógica actual del proyecto.
```

```
 */
```

```
public void setTalla(String nuevaTalla) {
```

```
    this.talla = nuevaTalla;
```

```
}
```

```
}
```

-Imagenes de Perro.java

```

package ejercicio1ud5ed;

/**
 * Clase que representa a un Perro e implementa la interfaz Animal.
 * Define el comportamiento de un canino, incluyendo su estado de ánimo y sonidos.
 *
 * @author Mateo Daniel Ortuño Ovando (mateoortuno11@gmail.com)
 * @version 1.0
 */
public class Perro implements Animal {

    /** El nombre del perro */
    private String nombre;

    /** Indica si el perro tiene un comportamiento agresivo o molesto en el momento */
    private boolean enfadado;

    /** Atributo para almacenar el tamaño del perro. @deprecated Este campo ya no se utiliza. */
    private String talla;

    /**
     * Constructor para la clase Perro.
     * Inicializa al perro con un nombre y un estado no enfadado por defecto.
     */
}

```

Activar Windows  
Ve a Configuración para activar Windows.

\_Hemos implementado comentarios @Override para hacer saber que los codigos estan reponiendose en nuestro fichero.

\_arrobas de @return y @deprecated, el deprecated como bien sabemos indica obsolescencia de un metodo en nuestro proyecto, no lo borramos sin embargo sirve poner este arroba para hacerlo saber.

## 2.3. Clase Ejercicio1UD5ED.java

\_fichero de Ejercicio1UD5ED.java:

```
/**
```

- Clase principal que actúa como punto de entrada del proyecto.
- Su función es instanciar objetos de tipo Perro y Gato para demostrar
- el funcionamiento de las clases y la interfaz Animal.
- 
- @author Mateo Daniel Ortuño Ovando ([mateoortuno11@gmail.com](mailto:mateoortuno11@gmail.com))
- @version 1.0

```
*/ public class Ejercicio1UD5ED {
```

```
/**
```

- \* Método principal (main) que ejecuta la lógica de la aplicación.
- \* Realiza las siguientes acciones:
- \* <ul>
- \* <li>Instancia objetos de las clases {@code Perro} y {@code Gato}.</li>
- \* <li>Modifica el estado de los objetos (ej. enfado del perro).</li>
- \* <li>Muestra por consola los atributos y sonidos de cada animal.</li>

```

* </ul>
*
* @param args Argumentos de la línea de comandos (no se utilizan en esta
implementación).
*/
public static void main(String[] args) {
    // Lógica de creación y visualización de animales
    // ...
    // Crear un perro
    Perro perro = new Perro("Fido");
    perro.setEnfadado(true);

    // Crear un gato
    Gato gato = new Gato("Garfield");

    // Mostrar información sobre el perro
    System.out.println("Nombre del perro: " + perro.getNombre());
    System.out.println("Sonido del perro: " + perro.hacerSonido());
    System.out.println("¿Está ladrando? " + perro.isLadrando());

    // Mostrar información sobre el gato
    System.out.println("Nombre del gato: " + gato.getNombre());
    System.out.println("Sonido del gato: " + gato.hacerSonido());
    System.out.println("¿Está ronroneando? " + gato.isRonroneando());
}

}

```

-imagen del fichero :

```

/**
 * Clase principal que actúa como punto de entrada del proyecto.
 * Su función es instanciar objetos de tipo Perro y Gato para demostrar
 * el funcionamiento de las clases y la interfaz Animal.
 *
 * @author Mateo Daniel Ortuño Ovando (mateoortuno11@gmail.com)
 * @version 1.0
 */
public class Ejercicio1UD5ED {

    /**
     * Método principal (main) que ejecuta la lógica de la aplicación.
     * Realiza las siguientes acciones:
     * <ul>
     * <li>Instancia objetos de las clases {@code Perro} y {@code Gato}.</li>
     * <li>Modifica el estado de los objetos (ej. enfado del perro).</li>
     * <li>Muestra por consola los atributos y sonidos de cada animal.</li>
     * </ul>
     *
     * @param args Argumentos de la línea de comandos (no se utilizan en esta impleme

```

\_En nuestro fichero main, como dato adicional, en el archivo html que genera javadoc, el fichero main tiene un apartado grande de recuadro y como el archivo es html, podemos usar atributos de html como un enlistado para que se vea mejor organizado <ul> <li>

\_incluimos el arroba @param para indicar paramtros del main (args)

\_Describimos que hace main y a que clases llama con @code.

### 3. GENERACIÓN DE JAVADOC (HTML)

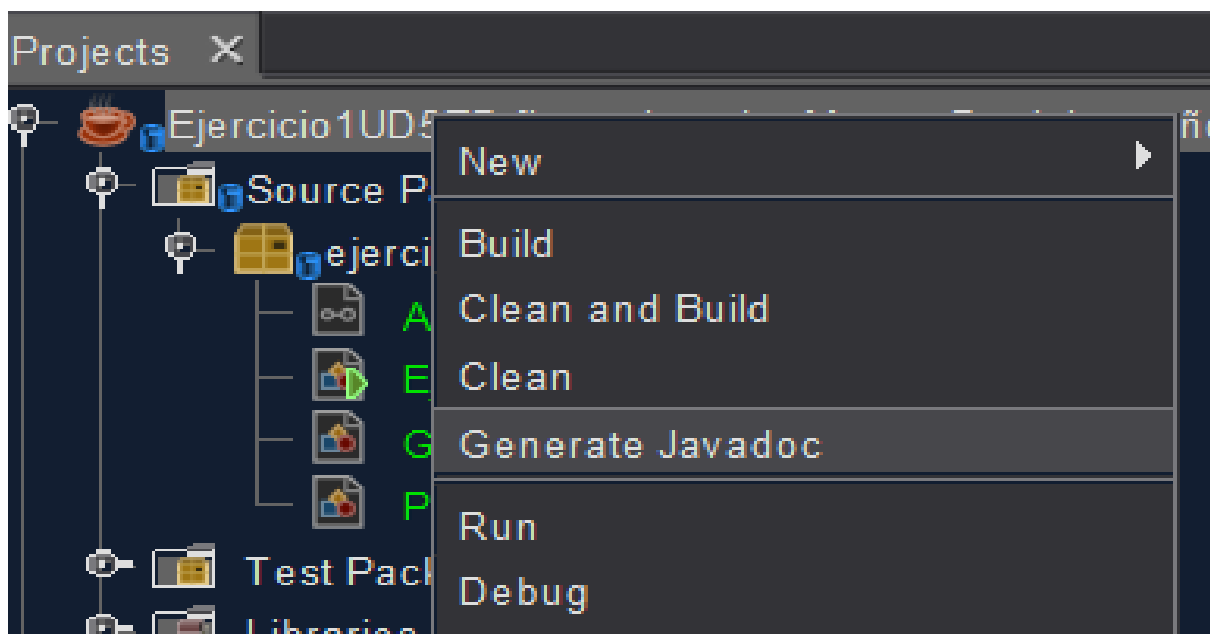
\_Vamos a indicar como hemos generado nuestro javadoc desde el ide netBeans.

### 3.1. Proceso de generación

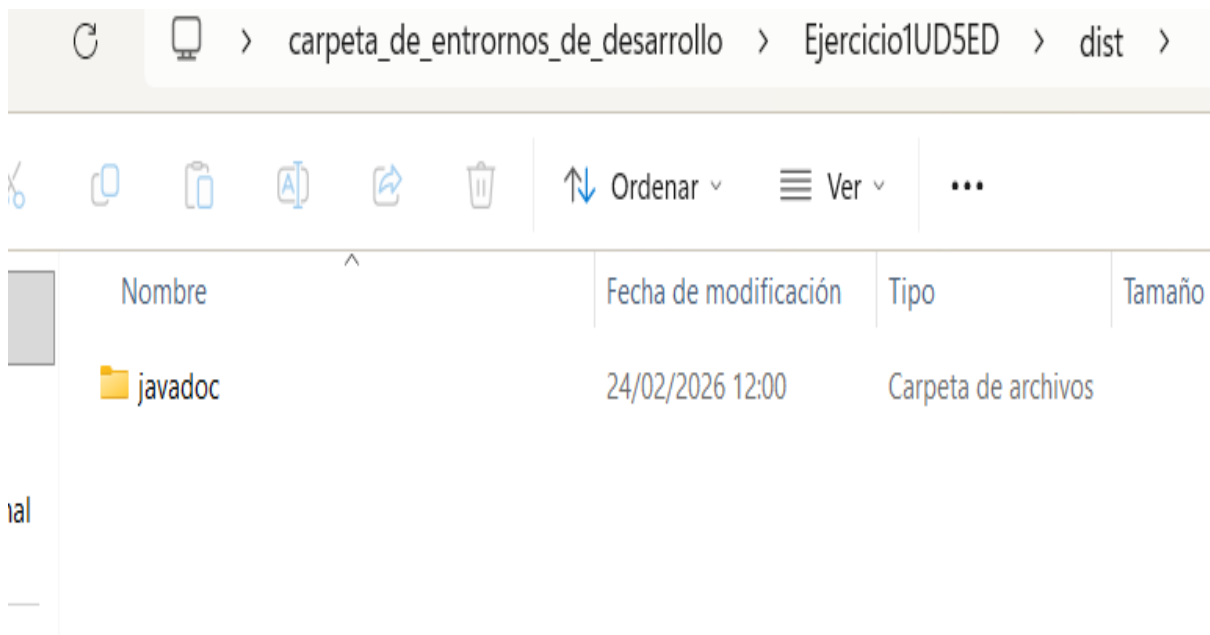
\_Primero nos dirigimos a la interfaz de netBeans, para luego dirigirnos en nuestros archivos a la izquierda que estan disponibles con nuestros proyectos.



\_En nuestro proyecto hacemos click derecho sobre el raton en el proyecto y le damos a generate javadoc.



\_Una vez se genere nuestro javadoc, Podremos ver que en nuestra carpeta de proyecto se ha creado un archivo llamado dist.



\_Aqui es donde se creo nuestro javadoc, aunque netBeans nos lo abre por defecto cuando lo crea, se almacena en la carpeta de nuestro proyecto.

|                         |                  |                     |       |
|-------------------------|------------------|---------------------|-------|
| ejercicio1ud5ed         | 24/02/2026 12:00 | Carpeta de archivos |       |
| index-files             | 24/02/2026 12:00 | Carpeta de archivos |       |
| legal                   | 24/02/2026 12:00 | Carpeta de archivos |       |
| resource-files          | 24/02/2026 12:00 | Carpeta de archivos |       |
| script-files            | 24/02/2026 12:00 | Carpeta de archivos |       |
| allclasses-index.html   | 24/02/2026 12:06 | Chrome HTML Do...   | 6 KB  |
| allpackages-index.html  | 24/02/2026 12:06 | Chrome HTML Do...   | 4 KB  |
| deprecated-list.html    | 24/02/2026 12:06 | Chrome HTML Do...   | 5 KB  |
| element-list            | 24/02/2026 12:06 | Archivo             | 1 KB  |
| help-doc.html           | 24/02/2026 12:06 | Chrome HTML Do...   | 13 KB |
| index.html              | 24/02/2026 12:06 | Chrome HTML Do...   | 2 KB  |
| member-search-index.js  | 24/02/2026 12:06 | JSFile              | 2 KB  |
| module-search-index.js  | 24/02/2026 12:06 | JSFile              | 1 KB  |
| overview-tree.html      | 24/02/2026 12:06 | Chrome HTML Do...   | 5 KB  |
| package-search-index.js | 24/02/2026 12:06 | JSFile              | 1 KB  |
| search.html             | 24/02/2026 12:06 | Chrome HTML Do...   | 4 KB  |
| tag-search-index.js     | 24/02/2026 12:06 | JSFile              | 1 KB  |
| type-search-index.js    | 24/02/2026 12:06 | JSFile              | 1 KB  |

\_Dentro estaran un monton de archivos hmtl, que estaran enlazados entre si dando el diseño que tiene laa api de Java.

### 3.2. Vista previa del portal de documentación

\_Vamos a ver como queda el index con toda la documentacion dada que genero netBeans, el javadoc:

A screenshot of a web browser displaying the Java documentation for the package 'ejercicio1ud5ed'. The browser's address bar shows the path 'C:/Users/kaexd/Desktop/carpetas\_de\_desarrollo/Ejercicio1UD5ED/dist/javadoc/ejercicio1ud5ed/package-summary.html'. The page has a dark blue header with navigation links: 'PACKAGE' (highlighted in orange), 'USE', 'TREE', 'DEPRECATED', 'INDEX', 'SEARCH', and 'HELP'. Below the header, the package name 'ejercicio1ud5ed' is displayed in a light blue bar, followed by a search input field with the placeholder text 'Search documentation (type /)'. On the left side, there is a sidebar with a 'Contents' section containing links for 'Description' (highlighted in light blue) and 'Classes and Interfaces'. The main content area features the title 'Package ejercicio1ud5ed' in bold. Below the title, the text 'package ejercicio1ud5ed' is shown. There are three tabs: 'All Classes and Interfaces' (highlighted in orange), 'Interfaces', and 'Classes'. A table follows with two columns: 'Class' and 'Description'. The table lists four entries: 'Animal' (Interfaz que define el comportamiento básico de un objeto de tipo Animal.), 'Ejercicio1UD5ED' (Clase principal que actúa como punto de entrada del proyecto.), 'Gato' (Clase que representa a un Gato e implementa la interfaz Animal.), and 'Perro' (Clase que representa a un Perro e implementa la interfaz Animal.).



Method Details

main

```
public static void main(String[] args)
```

Método principal (main) que ejecuta la lógica de la aplicación. Realiza las siguientes acciones:

- Instancia objetos de las clases **Perro** y **Gato**.
- Modifica el estado de los objetos (ej. enfado del perro).
- Muestra por consola los atributos y sonidos de cada animal.

Parameters:

args - Argumentos de la línea de comandos (no se utilizan en esta implementación).

Activar Windows  
Ve a Configuración para activar Windows.

\_Los detalles de nuestro main.

Method Summary

| All Methods       | Instance Methods              | Concrete Methods                                   |
|-------------------|-------------------------------|----------------------------------------------------|
| Modifier and Type | Method                        | Description                                        |
| String            | getNombre()                   | Recupera el nombre del gato.                       |
| String            | hacerSonido()                 | Emite el sonido característico del gato.           |
| boolean           | isRonroneando()               | Comprueba si el gato está ronroneando actualmente. |
| void              | setRonronea(boolean ronronea) | Permite modificar el estado de ronroneo del gato.  |

Methods inherited from class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

\_Nuestro metodo gato, que en cada get nos pilla el comentario que hicimos de su funcionalidad automaticamente en javadoc.

## Method Summary

| All Methods         | Instance Methods                                     | Concrete Methods                                                                 | Deprecated Methods |
|---------------------|------------------------------------------------------|----------------------------------------------------------------------------------|--------------------|
| Modifier and Type   | Method                                               | Description                                                                      |                    |
| String <sup>↗</sup> | <code>getNombre()</code>                             | Obtiene el nombre del perro.                                                     |                    |
| String <sup>↗</sup> | <code>getTalla()</code>                              | <b>Deprecated.</b><br>Este método ya no se usa en la lógica actual del proyecto. |                    |
| String <sup>↗</sup> | <code>hacerSonido()</code>                           | Emite el sonido característico de un perro.                                      |                    |
| boolean             | <code>isEnfadado()</code>                            | Consulta si el perro se encuentra enfadado.                                      |                    |
| boolean             | <code>isLadrando()</code>                            | Determina si el perro está ladrando basándose en su nivel de enfado.             |                    |
| void                | <code>setEnfadado(boolean enfadado)</code>           | Actualiza el estado de enfado del perro.                                         |                    |
| void                | <code>setTalla(String<sup>↗</sup> nuevaTalla)</code> | <b>Deprecated.</b><br>Este método ya no se usa en la lógica actual del proyecto. |                    |

Activar Windows

\_Lo mismo en la clase perro, pero haciendo incapie en los metodoso `@Deprecated`, que como dijimos son obsoletos o ya no se usan en el proyecto.

## 4. CONCLUSIÓN

\_ Tras la realización de esta práctica, se ha transformado un código fuente plano en un proyecto profesional, estructurado y fácil de mantener.

\_ Con la documentación se puede ver de un proyecto mucho más claro en el aspecto de el mantenimiento con ayuda de los apartados con `@deprecated`, esto hace ver más claro como puede evolucionar un proyecto.

\_ La documentación ayuda a otros programadores ver la lógica de los métodos, más que un requisito técnico una ayuda.